Exercise – Understanding the method of block bootstrap

1. Aim of the exercise

The aim of this exercise is to understand how to correctly estimate the sampling distribution of a statistic (e.g., the sample mean) when data are serially correlated, such as in time-series settings. Standard bootstrap methods assume independent observations and can lead to biased inference. We use the block bootstrap method to account for dependence across observations and compare its performance to sampling directly from the population.

2. Theory

Before beginning this application exercise, it is recommended to review the theory exercise to familiarize yourself with the underlying framework.

3. Set values for the parameters of the simulation

Clear all variables from memory and define the total number of simulations or samples to be generated.

```
%% 3. Set values for the parameters of the simulation

% 3.1. Clear the memory
clear;

% 3.2. Set the number of simulations as the number of bootstrap samples
N_sim = 5000;
```

4. Generate population data

Set the population size. Our aim is to generate an autoregressive process of order one, AR(1), defined by $X_t = \phi X_{t-1} + \epsilon_t$, where $\epsilon_t$ represents white-noise errors. This construction yields a stationary time series with serial dependence. To simulate the process, we generate i.i.d. shocks $\epsilon_t$ from a standard normal distribution, which serve as the driving noise. The persistence parameter $\phi$ controls the strength of temporal dependence, with higher values indicating stronger autocorrelation between successive observations.

```
%% 4. Generate population data

% 4.1. Set the population size
N_obs_pop = 100000;

% 4.2. Set AR(1) persistence parameter
phi = 0.7;

% 4.3. Generate white noise innovations
varepsilon = random('Normal',0,1,[N_obs_pop,1]);

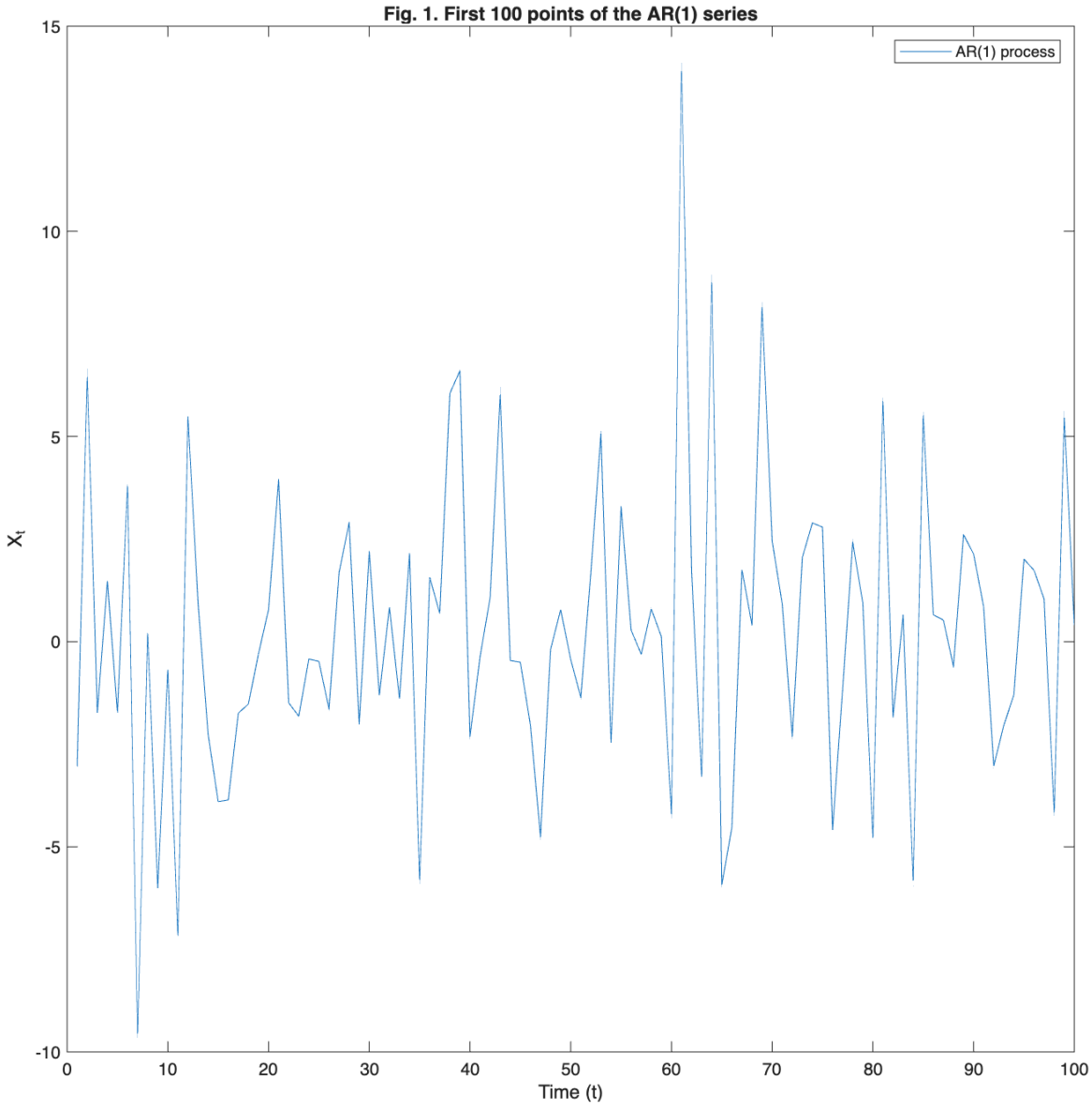
% 4.4. Simulate AR(1) time-series data
```

```
data_pop = filter(1,1-phi,varepsilon);
```

5. Plot AR(1) time series

Figure 1 presents the first 100 observations of the simulated AR(1) time series to illustrate the temporal structure and autocorrelation inherent in the population data. The series displays clear short-range dependence, characterized by local persistence and clusters of elevated or diminished values, an expected outcome given the autoregressive parameter $\phi = 0.7$. This pattern exemplifies the type of dependence for which the block bootstrap method is particularly well-suited.

```
%% 5. Plot AR(1) time series
figure('Position',[100 100 1000 1000]);
plot(1:100,data_pop(1:100));
xlabel('Time (t)');
ylabel('X_t');
title('Fig. 1. First 100 points of the AR(1) series');
legend('AR(1) process');
```


Fig. 1. First 100 points of the AR(1) series
AR(1) process
15
10
5
0
-5
-10
$X_t$
0
10
20
30
40
50
60
70
80
90
100
Time (t)

6. Draw samples from the population

In this bootstrap exercise, we focus on the sample mean as our statistic of interest and examine its sampling distribution under serial dependence. To evaluate how well the non-overlapping block bootstrap captures the uncertainty around the mean, we need a benchmark distribution for comparison. Since we have access to the full simulated AR(1) population, we can approximate the true sampling distribution by repeatedly drawing samples of size n directly from the population and computing their means.

Each sample is drawn as a contiguous block, a sequence of n consecutive observations, preserving the temporal structure and autocorrelation of the original time series. In each iteration, a random starting index is selected, and a block of $n$ adjacent values is extracted. The sample mean of each block is stored in the vector `means_data_samples_pop`, forming an empirical reference distribution. This benchmark allows us to assess how closely the bootstrap-generated distribution replicates the true sampling variability of the mean.

```
%% 6. Draw samples from the population

% 6.1. Set the sample size
N_obs_sample = 5000;

% 6.2. Preallocate matrix to store samples
data_samples_pop = NaN(N_obs_sample,N_sim);

% 6.3. Preallocate vector to store sample means
means_data_samples_pop = NaN(N_sim,1);

% 6.4. Sample from the population and compute the sample mean each time
for i = 1:N_sim
    start_idx = randi(N_obs_pop-N_obs_sample+1);
    data_samples_pop(:,i) = data_pop(start_idx:start_idx+N_obs_sample-1);
    means_data_samples_pop(i) = mean(data_samples_pop(:,i));
end
```

7. Pick an "initial" sample

To initiate the bootstrap procedure, we begin by randomly selecting one of the previously drawn contiguous samples from the population. This sample, of length $n$, represents the kind of empirical data typically available in real-world applications. We then compute the optimal block length l using the rule-of-thumb $l = \lceil n^{1/5} \rceil$, as proposed by Hall et al. (1995) for estimating two-sided distribution functions. Based on this block length, we calculate the number of full blocks $k = \lfloor n/l \rfloor$ that can fit within the sample. To ensure clean partitioning, we truncate the sample to its first $kl$ observations, discarding any leftover points that would prevent reshaping into equal-sized blocks. Finally, the trimmed sample is reshaped into $k$ sequential, non-overlapping blocks of size $l$, which serve as the input units for the block bootstrap procedure.

```
%% 7. Pick an "initial" sample

% 7.1. Randomly pick one sample index
```

```
sample_index = randi(N_sim);

% 7.2. Pick one sample from the previously drawn samples
data_sample = data_samples_pop(:,sample_index);

% 7.3. Compute optimal block length
block_length = ceil(N_obs_sample^(1/5));

% 7.4. Compute how many full blocks can be formed
N_blocks = floor(N_obs_sample/block_length); % Max. num. of full blocks

% 7.5. Trim sample to fit an exact number of full blocks
trimmed_sample = data_sample(1:N_blocks*block_length);

% 7.6. Reshape trimmed sample into non-overlapping blocks
blocks = reshape(trimmed_sample,block_length,N_blocks)';
```

8. Draw (bootstrap) samples from the initial sample

We begin by preallocating a vector to store the sample means generated by the bootstrap procedure. For each of the `N_sim` replications, we draw $k$ blocks with replacement from the original sample, which has already been partitioned into $k$ non-overlapping blocks of size $l$. These sampled blocks are concatenated into a single bootstrap sample by flattening them into a one-dimensional vector. The sample mean of this bootstrap sample is then computed and stored. Repeating this process across all replications yields an empirical bootstrap distribution of the sample mean, which can be compared to the benchmark distribution derived from the population.

```
%% 8. Draw (bootstrap) samples from the initial sample

% 8.1. Preallocate vector to store (bootstrap) sample means
means_data_samples_boot = NaN(N_sim,1);

% 8.2. Resample k blocks from the initial sample and compute the mean
for i = 1:N_sim
    data_samples_boot = datasample(blocks,N_blocks,1,'Replace',true);
    data_samples_boot_flat = data_samples_boot(:);
    means_data_samples_boot(i) = mean(data_samples_boot_flat);
end
```

9. Plot estimated PDFs

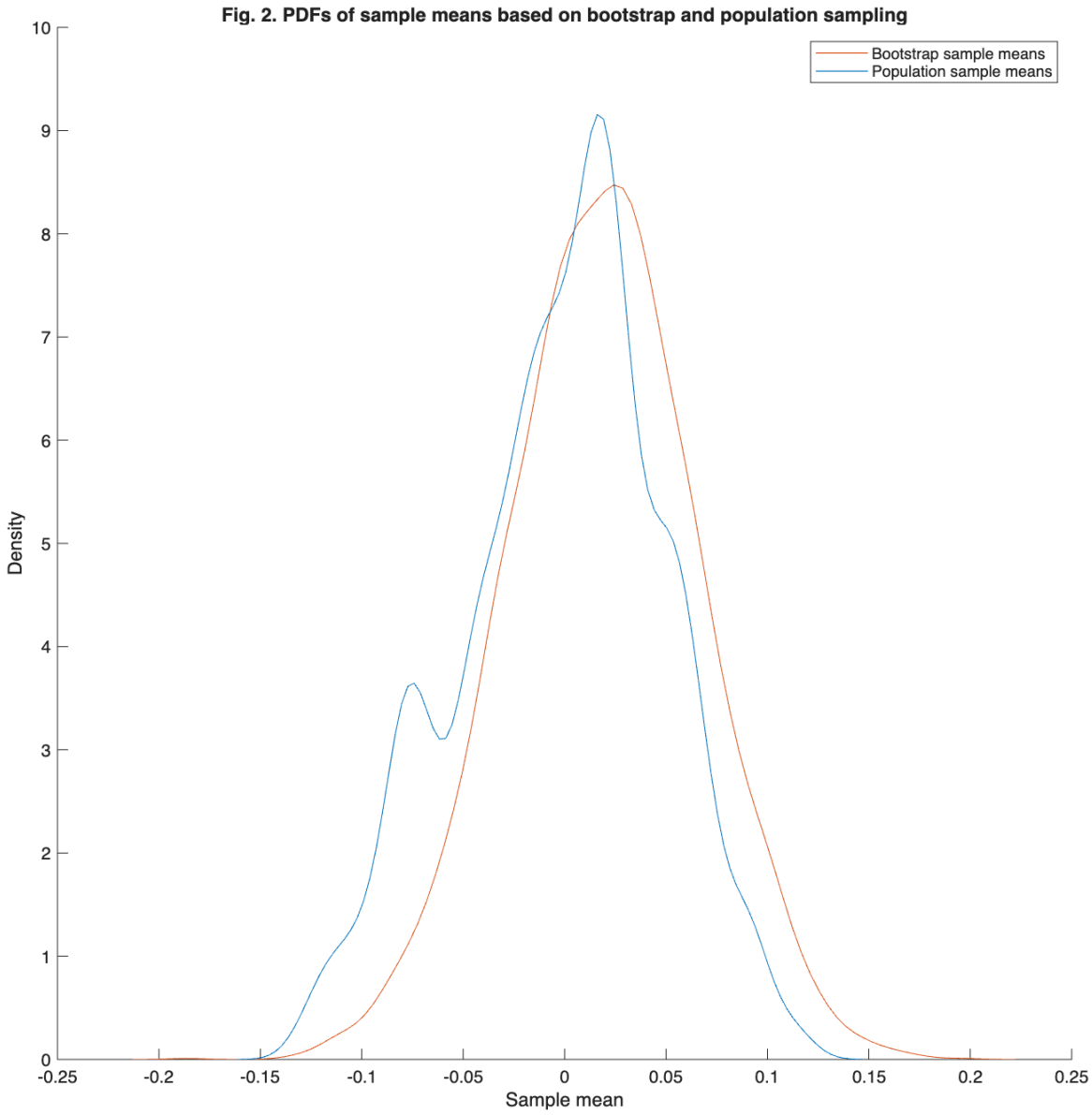
We use `ksdensity` to estimate the probability density functions (PDFs) of the sample means obtained from both the block bootstrap and population-based sampling procedures. The bootstrap distribution is based on resampling blocks from a single sample originally drawn from the AR(1) population, while the benchmark distribution is derived from repeated sampling directly from the full simulated population. Figure 2 shows that the block bootstrap yields a PDF that closely resembles the benchmark distribution, successfully capturing the variance inflation induced by serial dependence. However, a noticeable difference in smoothness emerges between

the two curves. This is a natural consequence of the sampling methods: the block bootstrap resamples from a single contiguous realization of the AR(1) process, which limits variability and results in a smoother distribution of sample means. In contrast, population sampling draws from the entire simulated time series, encompassing a wider range of behaviors and producing a more dispersed and less smooth density estimate. This contrast highlights a key theoretical distinction, resampling from a fixed path versus sampling from the full data-generating process, and illustrates how the block bootstrap can slightly underestimate variability while still preserving the dependence structure inherent in the data.

```
%% 9. Plot estimated PDFs

% 9.1. Estimate the density
[f_boot,x_boot] = ksdensity(means_data_samples_boot,'function','pdf');

% 9.2. Create a figure
figure('Position',[100 100 1000 1000]);
hold on
plot(x_boot,f_boot,'Color',[0.8500,0.3250,0.0980], ...
    'DisplayName','Bootstrap sample means');
[f_pop,x_pop] = ksdensity(means_data_samples_pop,'function','pdf');
plot(x_pop,f_pop,'Color',[0,0.4470,0.7410], ...
    'DisplayName','Population sample means');
title(['Fig. 2. PDFs of sample means based on bootstrap and ' ...
    'population sampling']);
ylabel('Density');
xlabel('Sample mean');
legend('show')
hold off
```




10. Plot estimated CDFs

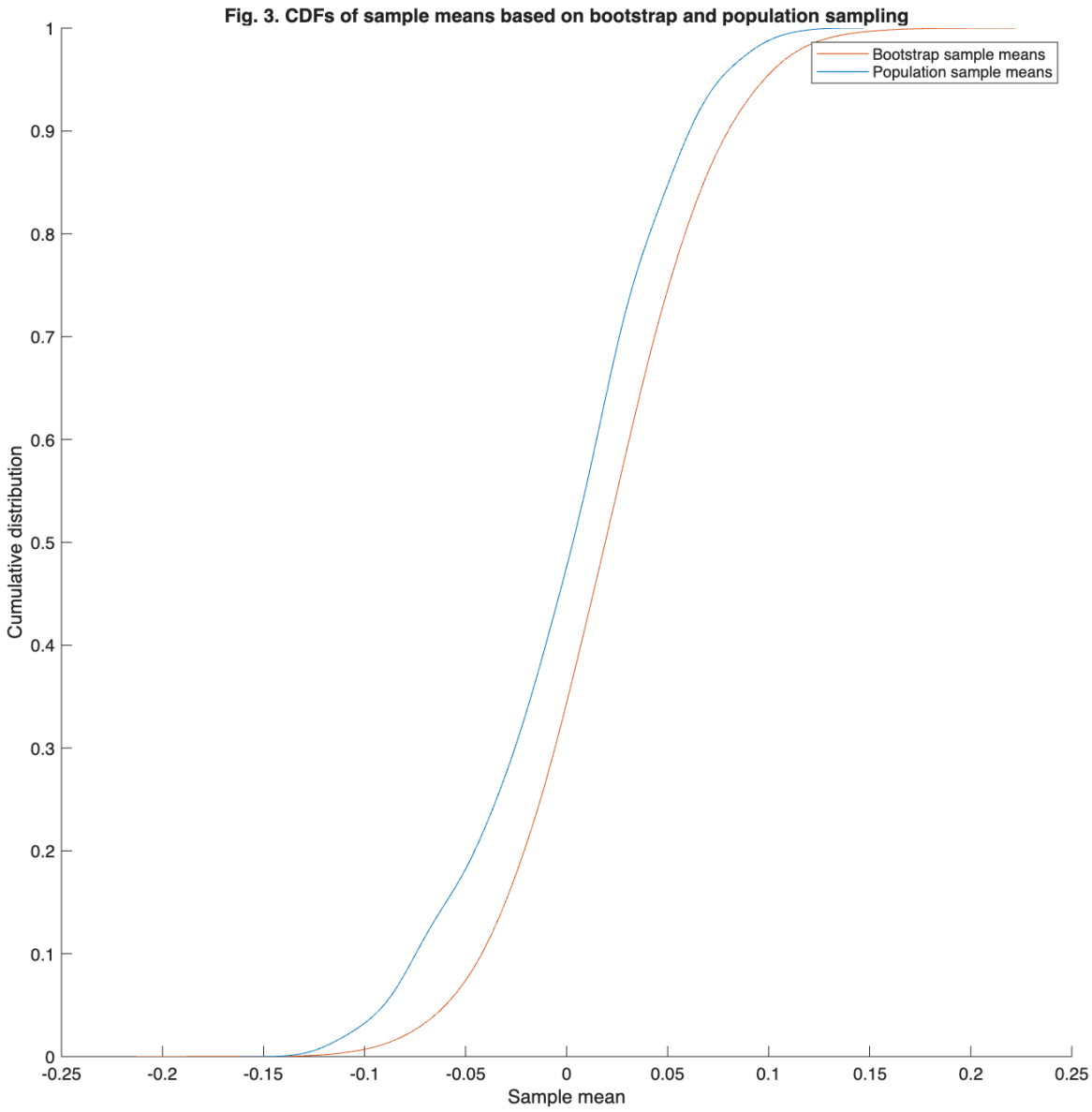
Figure 3 compares the empirical CDFs of the sample means obtained via block bootstrap and direct population resampling. The close alignment between the two CDFs indicates that the bootstrap distribution $\hat{G}_{n,m}(\tau, F_n)$ provides a reliable approximation of the benchmark distribution $\hat{G}_{n,m}(\tau, \mathcal{P})$, thereby confirming the practical validity of the non-overlapping block bootstrap procedure under time dependence.

```
%% 10. Plot estimated CDFs

% 10.1. Estimate the density
[f_boot,x_boot] = ksdensity(means_data_samples_boot,'function','cdf');

% 10.2. Create a figure
figure('Position',[100 100 1000 1000]);
hold on
```

```
plot(x_boot,f_boot,'Color',[0.8500,0.3250,0.0980], ...
    'DisplayName','Bootstrap sample means');
[f_pop,x_pop] = ksdensity(means_data_samples_pop,'function','cdf');
plot(x_pop,f_pop,'Color',[0,0.4470,0.7410], ...
    'DisplayName','Population sample means');
title(['Fig. 3. CDFs of sample means based on bootstrap and ' ...
    'population sampling']);
ylabel('Cumulative distribution');
xlabel('Sample mean');
legend('show');
hold off
```




## 6. Final notes

This file is prepared and copyrighted by Axel Zoons and Tunga Kantarcı. This file and the accompanying MATLAB file are available on GitHub and can be accessed using this link. This file has made use of four references. First is Radovanov and Marcikić, 2014. A comparison of four different block bootstrap methods. Croatian Operational Research Review. Second is Mignani and Rosa, 1995). The moving block bootstrap to assess the accuracy of statistical estimates in Ising model simulations. Computer physics communications. The third is Nordman and Lahiri, 2014. Convergence rates of empirical block length selectors for block bootstrap. The fourth is Hall, P., Horowitz, J. L., Jing, B. Y., 1995. On blocking rules for the bootstrap with dependent data. Biometrika, 82(3), 561-574.